

CONDITIONALS

(LECTURE)

REMINDERS

- HW00 is due tonight at 11:59pm
 - Use one late day --> submit by **1/29** @ 11:59pm
 - Use two late days --> submit by **1/30** @ 11:59pm
- Earn late tokens by handing in effortfully completed worksheets
- Check-in due before next class
- HW01 Released Tomorrow Afternoon, Due Weds
 - **START EARLY!**
- Recitation continues next week

HOW TO SUBMIT IN GRADESCOPE

FIRST SOME CATCHUP

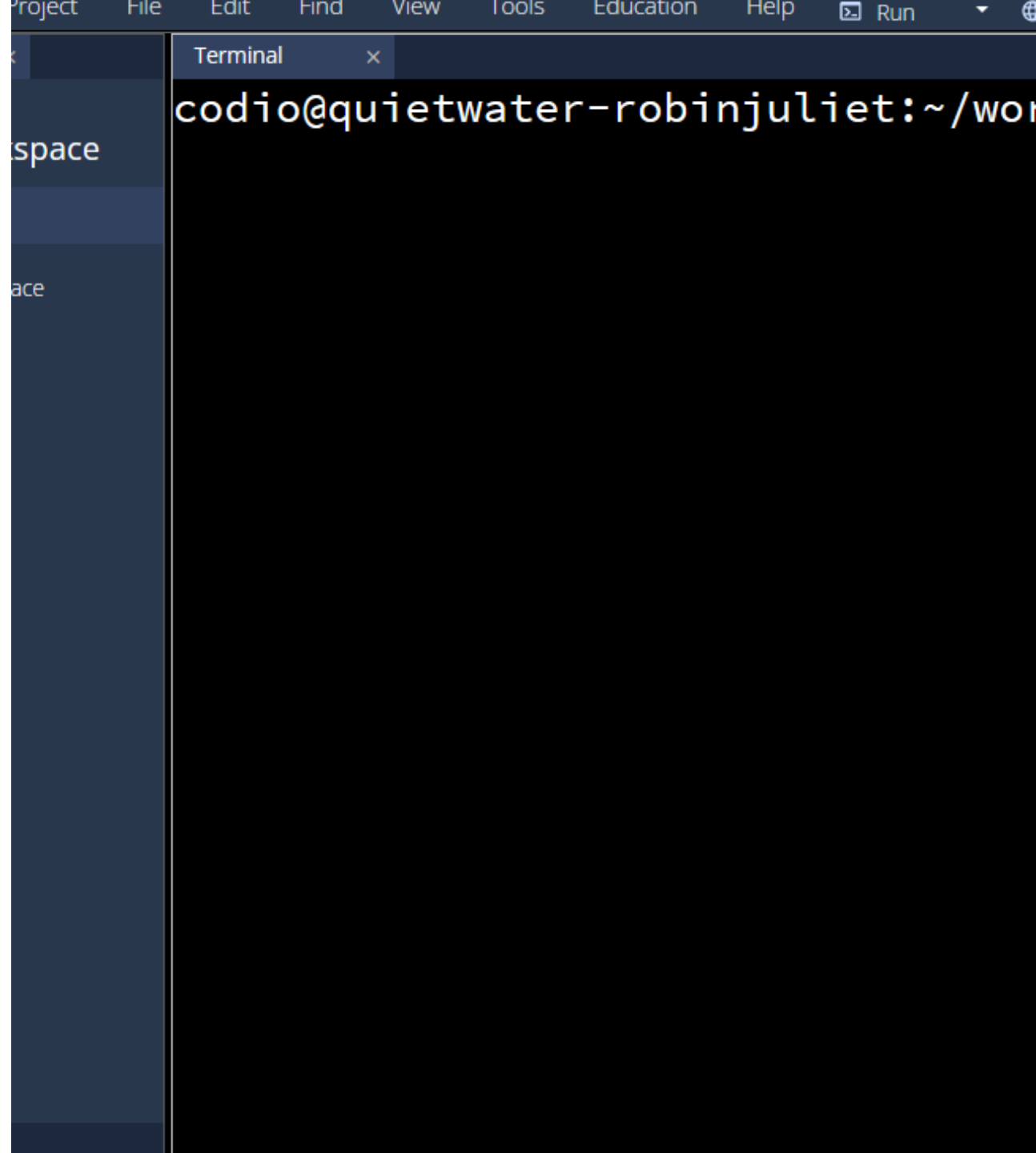
PYTHON REPL

Python has a REPL that you can use to run python code without storing it in a file.

REPL = **R**ead **E**valuate **P**rint **L**oop

Useful if you want to just check something real quick

To run, you can go to the "Run" or "terminal" tab in codio and type the command `python`



PYTHON REPL

To run, you can go to the "Run" or "terminal" tab in codio and type the command

```
python
```

Remember to end any running program first before you try to run the REPL

Once you have it open, you should be able to see it prompt you in the terminal:

PYTHON REPL

Once you have it open, you should be able to see it prompt you in the terminal:

I underlined the prompt in red

you can then type in lines of python and it will run those lines and remember variables.

if you don't assign a value into a variable, it will print that value.

Use `CTRL + D` or type `exit()` when done.

THE IN KEYWORD FOR STRINGS

In its simplest usage, it checks to see if `something` is within `something else`.

It evaluates to either `True` or `False`. As an example:

```
result = "pressure" in "under_pressure"  
print(result) # prints True!!
```

```
result = "CIS1100" in "CIS"  
print(result) # prints False!
```

```
result = "Pineapple" in "Pinecone"  
print(result) # also False!
```

**BACK TO OUR REGULARLY SCHEDULED
CONDITIONALS CONTENT**

QUICK RECAP: BOOLEAN EXPRESSIONS

Boolean expressions evaluate to `bool` values, i.e. either `True` or `False`.

We've looked at some cases where expressions will ALWAYS evaluate to `True` or `False`

e.g.

```
3 < 4 and 9 == (81 / 9)           # always True
not True and True or False and not False  # always False
```

But that's not super useful so....

BOOLEAN EXPRESSIONS WITH VARIABLES

We are also able to write boolean expressions that contain variables.

```
x % 3 == 2 and x > 5
```

not always True or False!

This expression's value changes based on the value of x!

REMINDER: THE BOOLEAN EXPRESSION TOOLKIT

Relational Operators:

OPERATOR/METHOD	INPUT TYPES	DESCRIPTION
< / <=	int, float, str	less than / less than or equal to
> / >=	int, float, str	greater than / greater than or equal to
== / !=	int, float, str	equal to / not equal to

THE BOOLEAN EXPRESSION TOOLKIT

Logical Operators:

OPERATOR/METHOD	INPUT TYPES	DESCRIPTION
and	bool	evaluates to True only if both inputs are True
or	bool	evaluates to True as long as at least one input is True
not	bool	negates a single bool value to its opposite

ACTIVITY: UNDER PRESSURE

M1:

I'm writing a program to monitor valve pressure in a chemical plant. I want to define *safe conditions* as those where the pressure is 0.5, 3.5, or between those two values. Which is a boolean expression that is True only when conditions are safe?

- (A) $0.5 < \text{pressure} \text{ and } 3.5 < \text{pressure}$
- (B) $0.5 \leq \text{pressure} \text{ and } 3.5 \geq \text{pressure}$
- (C) $0.5 \leq \text{pressure} \text{ or } 3.5 \geq \text{pressure}$
- (D) $0.5 \geq \text{pressure} \text{ or } 3.5 \leq \text{pressure}$
- (E) $0.5 > \text{pressure} \text{ and } 3.5 \leq \text{pressure}$

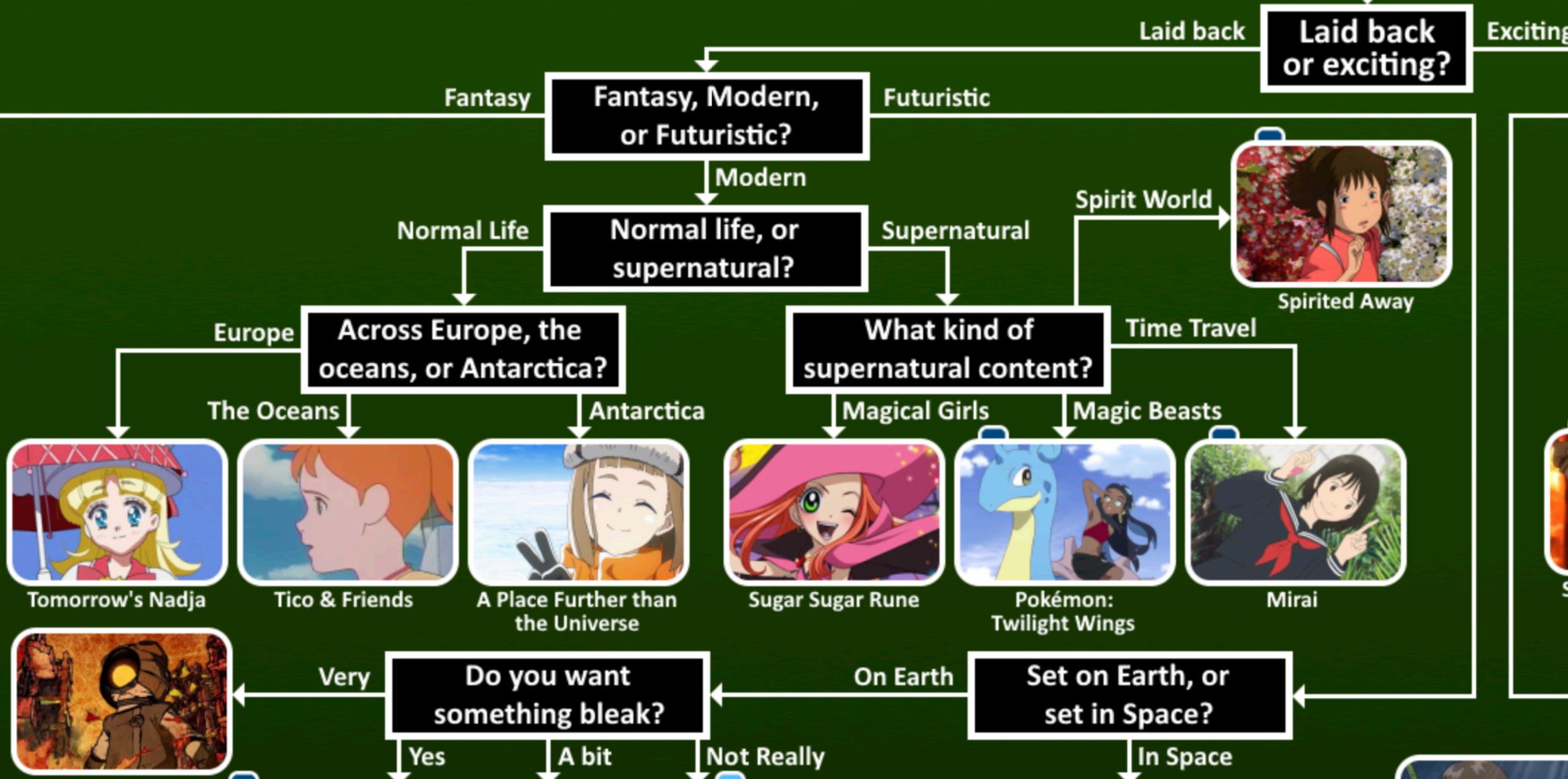
RECAP: IF

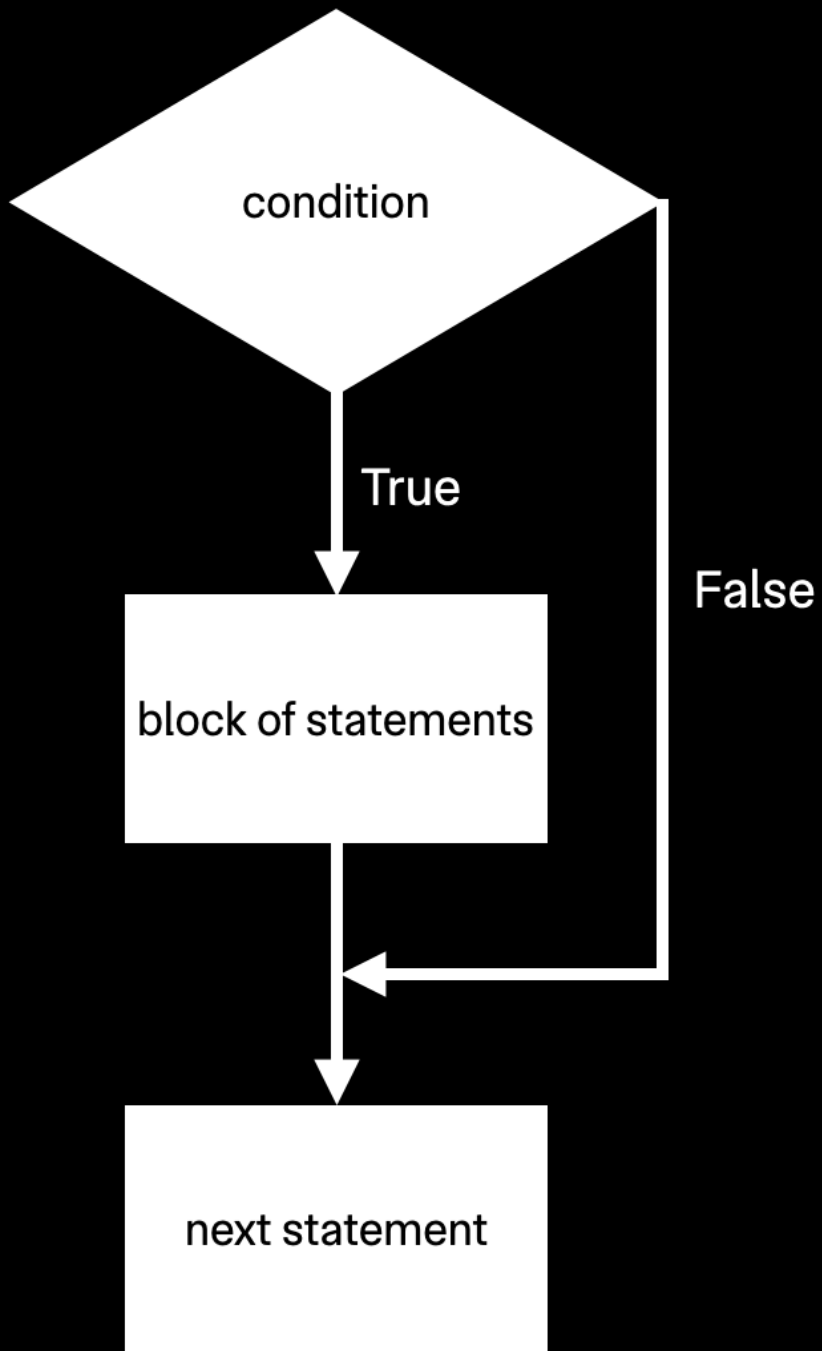
The `if` statement allows us to specify a portions of our program that should be run **only in the case that** a certain condition is met.

```
if my_boolean_expression:  
    statement_one  
    statement_two  
    ...  
    statement_last
```

```
if it_is_raining:  
    I will grab my umbrella  
I will go outside
```

Adventure





RECAP: CONTROL FLOW & IF

- Test the condition...
 - **if** it is `True`, execute the block of statements
 - otherwise, proceed to the next statement.

ACTIVITY:

```
import penndraw as pd
r = 0.1
if r == 0.1:
    pd.set_pen_color(pd.RED)
pd.set_pen_color(pd.GREEN)
pd.filled_circle(0.5, 0.5, r)
pd.set_pen_color(pd.BLACK)
```

M2: What color is the circle that gets drawn?

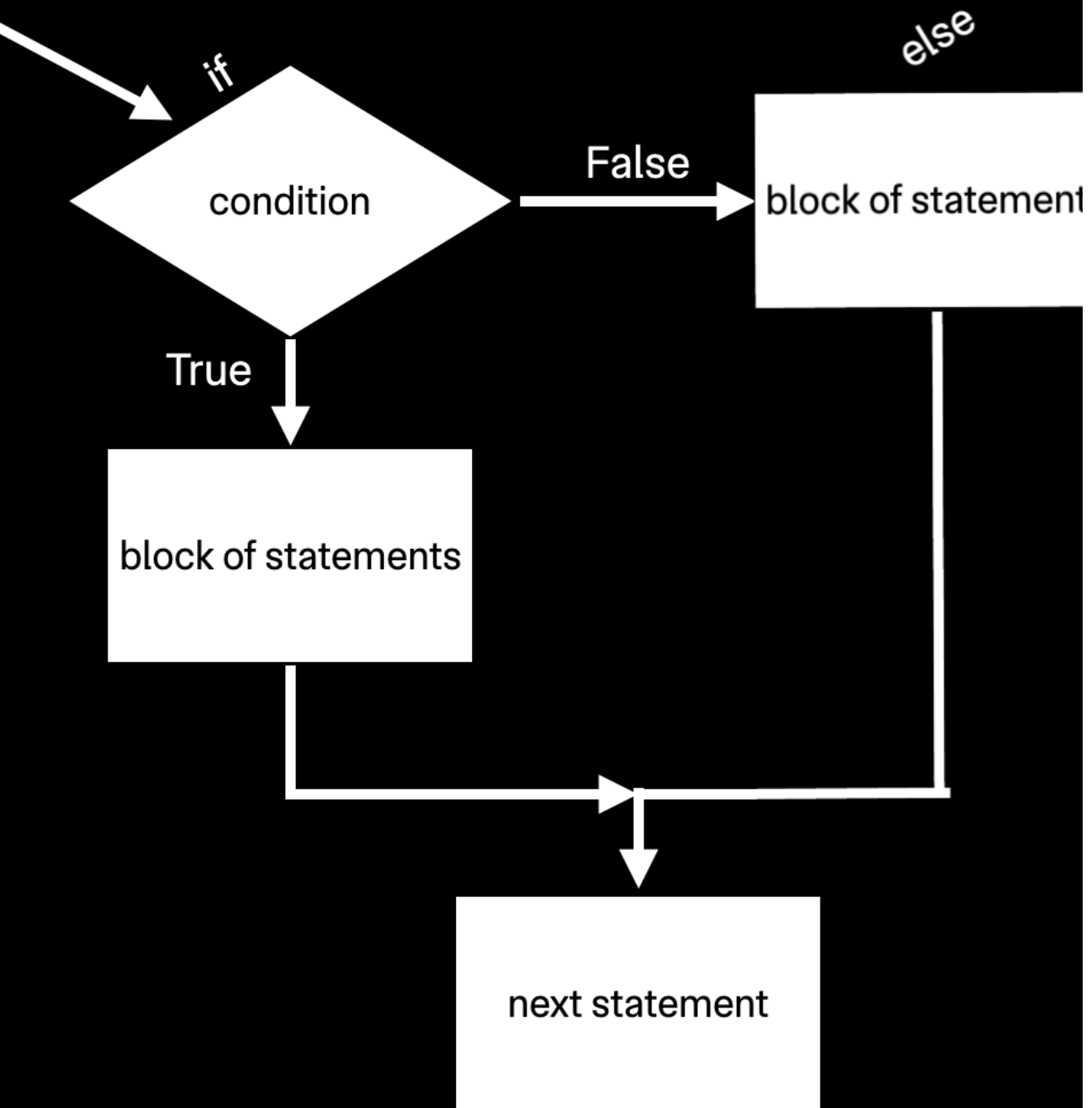
- (A) Red
- (B) Green
- (C) Black
- (D) There is no circle drawn

RECAP: ELSE

The `else` keyword allows us to define a body of statements that will be run if the condition was not met.

```
if boolean_expression:  
    block_one  
else:  
    block_two
```

```
if it_is_raining:  
    I will grab my umbrella  
else:  
    I will grab my hat  
I will go outside
```



RECAP: ELSE

Activity **C12**

Draw the flow diagram for the following "code":

```
if it_is_raining:  
    I will grab my umbrella  
else:  
    I will grab my hat  
I will go outside
```

MULTIPLE CONDITIONS

Sometimes we want to make multiple checks for things being true:

```
if it_is_raining:
    I will grab my umbrella
else:
    if it_is_snowing:
        I will grab my snow boots
    else:
        if it_is_sunny:
            I will wear sunscreen
        else:
            I will grab my hat
I will go outside
```

Let's try drawing this diagram on the board!

SO WHAT IS THE PROBLEM WITH THIS "CODE"?

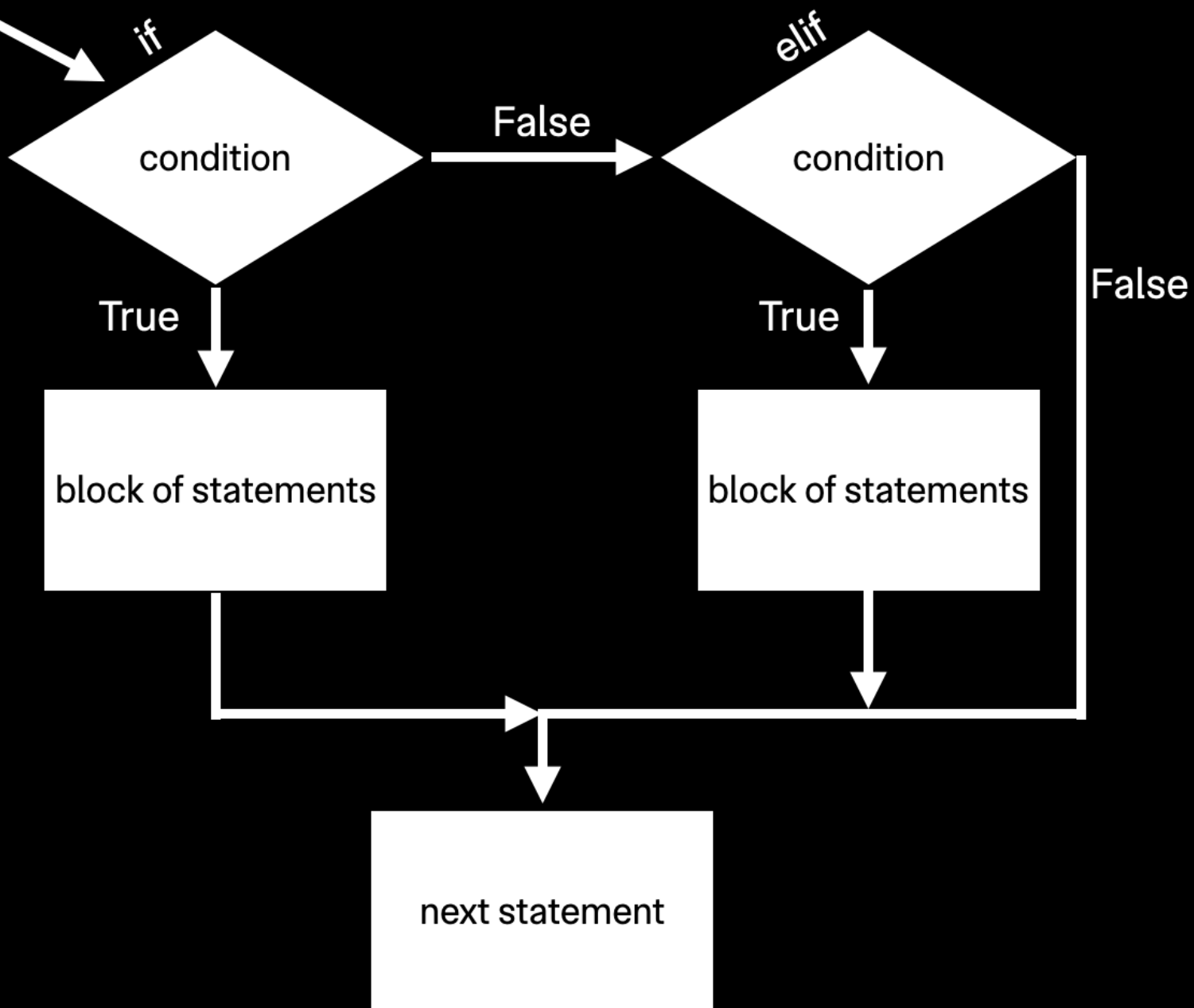
```
if it_is_raining:  
    I will grab my umbrella  
else:  
    if it_is_snowing:  
        I will grab my snow boots  
    else:  
        if it_is_sunny:  
            I will wear a hat  
I will go outside
```

RECAP: ELIF

`elif` allows you to specify an *alternative condition* that is be tested *only when all previous conditions were `False`*.

The `elif` syntax:

```
if first_boolean_expression:
    statement_one
    statement_two
    ...
    statement_last
elif alternative_boolean_expression:
    statement_a
    statement_b
    ...
    statement_z
```



RECAP: ELIF

`if` and `elif` statements represent *mutually exclusive* choices: we may execute the body of one, the other, or neither, but *never both*.

```
if it_is_raining:
    I will grab my umbrella
else:
    if it_is_snowing:
        I will grab my snow boots
    else:
        if it_is_sunny:
            I will wear a hat
I will go outside
```

is equivalent to

```
if it_is_raining:
    I will grab my umbrella
elif it_is_snowing:
    I will grab my snow boots
elif it_is_sunny:
    I will wear a hat
I will go outside
```

ACTIVITY: PICKING PATHS

C14: Draw a flow-chart representing the control flow of this program.

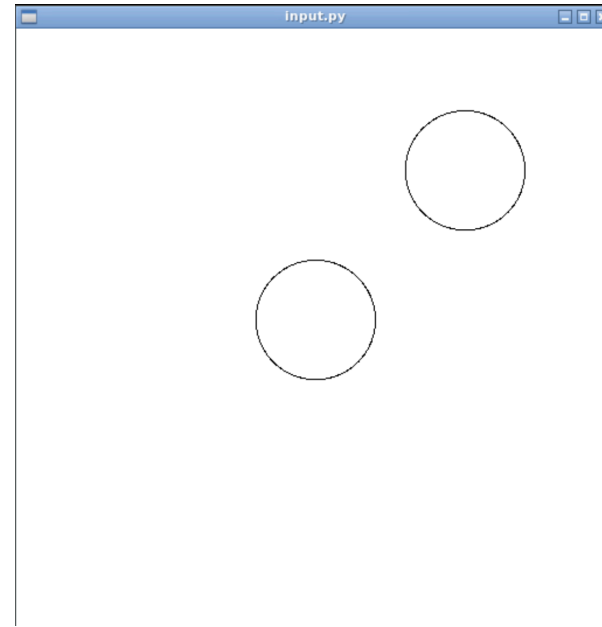
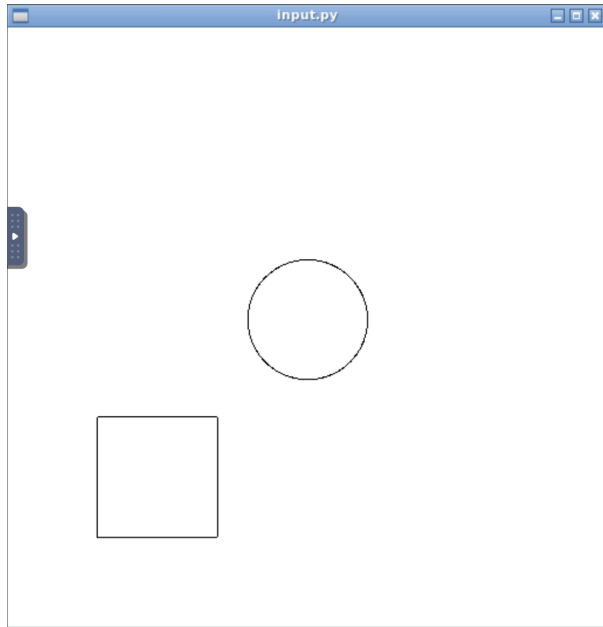
```
import penndraw as pd
code = input("TYPE THE CODE")

if len(code) == 4:
    pd.square(0.25, 0.25, 0.1)
elif not code.isalpha(): #This evaluates to True if all characters are alphabetical...
    pd.circle(0.75, 0.75, 0.1)

if code.endswith("!"): #This evaluates to True if it ends with the str inside ( ).
    pd.circle(0.5, 0.5, 0.1)
pd.run()
```

ACTIVITY: PICKING PATHS

From left to right, write the input <"CODE"> that a person could type in to generate each output. **L11**



RECAP: ALL CONDITIONALS

Recipe for any conditional:

1. Always start with an `if`. Each `if` comes with a boolean expression to test. This expression is always tested.
2. As many `elif` statements as desired. Each comes with a boolean expression. Each expression only tested if all previous are `False`.
3. An `else` statement, or not. No boolean expression provided. Body executed if all previous expressions are `False`.

ACTIVITY: FIX IT!

Both snippets below are broken for some reason. In **C16**, write an explanation for what is wrong in both cases.

```
x = int(input())
if x > 12:
    print("daylight")
elif x < -10:
    print("fires")
elif x != 45:
    print("ridges")
else x % 13 == 2:
    print("green")
```

```
y = input()
if y == "bliss":
    z = "ful"
else:
    print(y + z)
```

RECAP: CASE/MATCH: ANOTHER WAY TO CHOOSE

```
match traffic_light:
    case "red":
        print("Stop!")
    case "yellow":
        print("Slow down.")
    case "green":
        print("Proceed carefully.")
```

- `match` allows us to compare an expression's value to several different cases.
- Each `case` gives a value to compare to and what to execute if there's a match.
- Use `|` to specify multiple options per case
- Use `_` to specify a fall-back

ACTIVITY

How would you write this case/match as an if/elif/else statement? **C16**

```
match traffic_light:
    case "red":
        print("Stop!")
    case "yellow":
        print("Slow down.")
    case "green":
        print("Proceed carefully.")
```

ACTIVITY: TIER LIST

A *tier list* is an assignment of letter grades to different options. CIS 1100 TAs are really opinionated about lots of things, including different kinds of milks. Here are my personal rankings...:

MILKS	TIER
Oat, Whole Cow	S (highest)
2% Cow, Soy	A
Coconut, Condensed	B
All other milks	C
Almond, Goat	F (lowest; a disgrace)

```
milk = input()
match milk:
    case "Oat" | "Whole Cow":
        print("S")
    case "2% Cow" | "Soy":
        print("A")
    case "Coconut" | "Condensed":
        print("B")
    case "Almond" | "Goat":
        print("F")
    case _:
        print("C")
```




pixelatedboat aka "mr bluesky" @pixelatedboat.bsky.social · 2h

Nice try but this supposed "video" is just a series of still frames played fast enough to trick the eye into perceiving motion



Paul Frazee @pfrazee.com · 3h

here it is, the video that famously crashed the servers by going viral in 1895

